

Task Document: Task Management Application

Backend: Node.js + MongoDB | Frontend: Angular



Objective

Develop a complete Task Management application with secure backend APIs and a fully functional Angular frontend.

The solution must focus on authentication, data validation, UI/UX clarity, and clean coding principles.

1. User Authentication (Backend)

The authentication system must:

- Support new user registration
- Validate email uniqueness
- Hash passwords securely
- Provide a login endpoint returning Access and Refresh tokens
- Implement token refreshing logic
- Implement logout to invalidate refresh tokens
- Support user roles: user and admin
- Protect all task routes using authentication middleware

2. Task Management (Backend)

Each Task must store:

- ID
- Title
- Description
- DueDate
- Priority (Low/Medium/High)
- Status (Pending/In-Progress/Completed)
- Tags (Array)
- Files metadata (Array)
- CreatedBy (User ID)

- CreatedAt / UpdatedAt

Rules:

- Only authenticated users can access task endpoints
- Only the creator can update/delete their tasks
- Admin can view/modify/delete any task

3. Detailed Validations & Error Handling

All validations must be strictly implemented:

- Title is required and cannot be empty
- DueDate must be a valid **future** date
- Priority must match allowed values: Low / Medium / High
- Status must match allowed values
- Tags must be an array of text values
- Incoming file types must be validated (if provided)

Error Handling Requirements:

- Return structured error messages with HTTP codes
- Handle missing fields, malformed tokens, and unauthorized access
- Handle non-existent tasks and forbidden access attempts
- Return consistent JSON error format

4. File Upload Requirements

- Support uploading one or multiple files
- Store filenames, path, size, and mimetype
- Reject unsupported file formats
- Save files either locally or in a structured folder
- Associate uploaded files with their tasks

5. Email Notification System

Every time a new task is created:

- Send a formatted email to the admin email address
- Email must include:
 - Task title
 - Priority
 - Due date
 - Created by (user email)
- Email must follow a clean and readable format

6. Pagination, Filtering, Sorting & Search

Task list endpoint must support:

- Pagination → page & limit
- Filter by:
 - Priority
 - Status
 - DueDate range (from - to)
- Sorting options (e.g., newest first)
- Search support on title and description using partial text matching
- Combine multiple filters simultaneously

7. Task Activity Logs (Mandatory)

Create a separate "task_logs" collection to track all actions.

Each log must include:

- taskId
- userId
- action (Created / Updated / StatusChanged / Deleted)
- timestamp

Logs must be created automatically whenever a task is modified.

8. Rate Limiting Requirements

- Apply global rate limiting (e.g., 100 requests/min per IP)
- Return proper error message when limit exceeded

9. Frontend (Angular) Requirements

The Angular application must include:

Authentication UI:

- Registration form
- Login form
- Token stored securely (never in plain localStorage without protection)
- Route guard for protected pages
- Logout functionality

Task Module UI:

- Create Task Form
- Edit Task Form
- Task List Page with:
 - Pagination

- Filters
- Search
- Sorting
- Task Detail Page
- File Upload in forms
- Display file list in task details
- Show validation errors clearly

10. API Documentation Requirements

Backend must include Swagger documentation showing:

- Auth routes
- Task routes
- Sample request bodies
- File upload endpoints
- Query params for filters & pagination

11. Submission Guidelines

- Complete the task within 4 hours
- Publish backend + frontend on GitHub
- Add collaborator: nehawts
- Include a detailed README containing:
 - Setup instructions
 - Environment variables format
 - How to run backend & frontend
 - Testing instructions

End of Document